Local Coding Agents on One Workstation, Second Night

What the Serving Stack, the Prompt and the Model Each Contribute

Matthias Nott

Independent Research

Lab Report

Paired measurements of 15–16 October 2026

16 October 2026

Abstract

The first night showed that open-weight models on one workstation could not yet replace a cloud worker on real application fixes, but its comparisons were run in blocks and its local servers were still defective. This second night repeats the question with paired arms: the two arms of a comparison start together on the same task, only one model server is loaded at a time, and the local runs are graded by the hidden tests of the fixes that actually landed. Three findings stand out. First, naming the file and the function (the *precise* prompt) lifted the local Qwen3-Coder-30B-A3B worker from 1 of 9 to 5 of 9 passes against the orchestrator's specification; all four discordant pairs favoured the precise prompt (exact sign test $p = 0.125$, two-sided, which is the smallest value nine pairs can give with four discordant). Second, serving still decided the outcome: a second model family ran through a multimodal code path that never hit the prefix cache, so every turn re-read 40–50k tokens; after the fix, a smoke run fell from 55–61 s to 23–37 s and a one-function fix ran within about twice the time of the cloud reference. Third, a cloud plan written from the parent commit let three different local models pass a task that none of eight local precise or specification runs passed. A hardware watchdog reset at 23:43, with about 117 GB of model servers resident on a 128 GB machine, set the constraint for the rest of the night: one model server at a time. The samples are small and every statement is conditional on them. The final ranking is Sonnet 5.5 (4 of 4) ahead of GLM-5.3-flash and GLM-5.3 (3 of 4 each), Haiku 4.5 (2 of 4) and then the local models, of which Ornith-1.5-35B-A3B and Qwen3-Coder-30B-A3B are level on passes (7 of 16 and 4 of 8) and the dense Qwen3.8-27B is too slow to use; no local model passed the three-file task.

Contents

1 Introduction **4**

2 What Changed Since Night 1 **4**

3 Setting **4**

4 Method **5**
4.1 Tasks . . . 5
4.2 Three prompts, and a plan . . . 5
4.3 Grading and pairing . . . 5

5 Results **6**
5.1 Precise against specification on one local model . . . 6
5.2 Cloud yardsticks . . . 6
5.3 Local models, one at a time . . . 6
5.4 Serving findings . . . 8
5.5 Failure classes and levers . . . 9

6 The Reboot **9**

7 Discussion **10**

8 Recommendation **10**

9 Limitations **11**

References **12**

List of Tables

1 Models in the second night . . . 5
2 local30, specification against precise prompt (paired; minutes / turns where recorded) . . . 6
3 Precise prompt, cloud yardsticks (minutes to finish) . . . 7
4 Local models by prompt (night 2; minutes, turns where recorded; 20 or 40 min is the deadline; passed with median wall minutes of graded arms) . . . 7
5 Serving defects of night 2, before and after . . . 8

1 Introduction

The first report measured whether open-weight models on a single 128 GB workstation could act as the workers in an agentic coding workflow (Anthropic, 2026a). Its answer was no for real application fixes, but it left three questions open: whether the failures were a matter of finding the code or of changing it, whether the local servers were themselves the bottleneck, and whether the blocked order of the runs had biased the comparison.

The question for this report is narrower and more practical: *can a local coding setup run beside normal work?* That means one model server of moderate size, no extra hardware, and a worker that completes small, well-localised fixes in a time the operator will accept, while the machine is still used for other things. The cloud models are yardsticks only; the aim is a local solution, in speed and in quality.

Each experiment therefore answers one of three questions: which *serving setup*, which *prompt*, or which *model* makes a local worker pass more often and faster. Section 2 describes what was changed since the first night, Section 3 the setting, Section 4 the method, and Section 5 the results; Section 6 reports the reboot that constrained the night.

2 What Changed Since Night 1

Paired arms. The two arms of a comparison run on the same task and start together, so thermal state, load and the build queue affect both alike. Night 1 ran its arms in blocks.

One model server at a time. After the reboot of Section 6, models are compared phase by phase on an otherwise idle machine; within a phase, the paired arms share the server.

Path repair. Local models repeatedly mistyped absolute paths into the arm's own clone, one character off from the working directory, and received "file does not exist" again and again. A worker-only hook now repairs a non-existent absolute path when its prefix is within three edits of the working directory and the remainder exists below it. Arms started after the hook was registered use it; both arms of a pair start together, so pairs remain comparable.

Isolation. As on night 1, each run starts in its own clone with only the parent commit's history; hidden tests are overlaid at grading time. Every row is tagged with the night so the two nights stay apart.

3 Setting

All measurements ran on one Apple MacBook Pro with an M5 Max and 128 GB of unified memory, set to maximum performance by the operator; the thermal level and the load average were logged with every pair. Memory pressure and swap use were logged every 10 s during runs. Local models were served with vllm-mlx (vllm-mlx contributors, 2026) through patched

launchers. Workers were started through PAI (Nott, 2026b); sessions exchanged messages through AIBroker (Nott, 2026a). The workloads are anonymised: Project A is the native PDF application that supplies the coding tasks, Project B the document-management system that shares the machine.

Table 1: *Models in the second night*

Model	Placement	Size	Role
Qwen3-Coder-30B-A3B (Qwen Team, 2025)	local	4-bit	Routed local model (“local30”)
Ornith-1.5-35B-A3B	local	4-bit	New local candidate
Qwen3.8-27B	local, dense	6-bit	New local candidate (“qwen38”)
GLM-4.7-Flash (Z.ai, 2026)	local	6-bit	New local candidate (“glm47”)
Claude Sonnet 5.5 (Anthropic, 2026b)	cloud	–	Yardstick
Claude Haiku 4.5	cloud	–	Yardstick
GLM-5.3	cloud	–	Yardstick
GLM-5.3-flash	cloud	–	Yardstick

4 Method

4.1 Tasks

The tasks are the replay tasks of night 1: fixes that already landed in Project A, started from the parent commit and graded by the hidden tests that came with the fix. Four tasks have a precise prompt and were used tonight: *log-size* (one function, +21 lines in the landed fix), *box-press* (2 files, +79), *view-decimals* (1 file, +6) and *save-model-ref* (3 files, +64).

4.2 Three prompts, and a plan

The *task* prompt is the maintainer’s symptom in their own words. The *spec* prompt is what the orchestrator wrote when it delegated the fix: the diagnosis and the owning part of the code, without the fix. The *precise* prompt names the exact file and function. The *plan* prompt adds an edit plan (files, functions, what to change, no code) written by a cloud model from the parent commit without access to the fix: “cloud head, local hands”. A leak audit checked that no prompt contained the fix.

4.3 Grading and pairing

A patch passes when the result builds and the hidden tests pass; no model judges another. Wall time is censored at the task deadline. Pairs are two arms of the same task started together; discordant pairs are counted and an exact sign test is applied. Wilson intervals

(Wilson, 1927) are given for single arms. Tonight's samples are one to three repeats per task and cell.

5 Results

5.1 Precise against specification on one local model

Table 2 gives the nine completed pairs for the local30 model. Precise passed 5 of 9 (Wilson 27–81 %), the specification 1 of 9 (2–44 %). Of the nine pairs, four were discordant, all in favour of precise; the exact sign test gives $p = 0.125$ two-sided, which is the smallest value attainable with four discordant pairs. The two tasks log-size and box-press, where the precise prompt names the function to change, account for all the gain; precise runs also used about half the turns. view-decimals and save-model-ref failed in both arms in every repeat: for view-decimals the worker reached the right file and the change itself was wrong, so those failures are about making the change, not finding it.

Table 2: *local30, specification against precise prompt (paired; minutes / turns where recorded)*

Pair	Spec	Precise
log-size, rep 1	fail 14.6 / 41	**pass** 11.6 / 27
box-press, rep 1	fail 14.7 / 37	**pass** 15.2 / 33
view-decimals, rep 1	fail 6.2 / 40	fail 2.2 / 15
save-model-ref, rep 1	fail 18.4	fail 12.9
log-size, rep 2	fail 13.0 / 79	**pass** 6.6 / 27
box-press, rep 2	fail 15.5 / 52	**pass** 15.2 / 32
view-decimals, rep 2	fail 5.8 / 35	fail 13.4 / 53
save-model-ref, rep 2	fail 14.2 / 35	fail 14.1 / 31
log-size, rep 3	**pass** 8.8 / 40	pass 6.2 / 28
Passed	1 / 9	5 / 9

5.2 Cloud yardsticks

Table 3 gives the cloud models on the precise prompt, one agent at a time, on an otherwise idle machine (the GLM-5.3 run was made on a loaded machine). save-model-ref, which changes three files and adds 64 lines, is the task that separates Sonnet 5.5 from all others; Sonnet takes 6 to 7 minutes on the two tasks that no local model has passed. On the specification prompt GLM-5.3 passed 3 of 4 as well (log-size, box-press and, after 39.6 minutes and 117 turns, save-model-ref, graded by hand), failing view-decimals; with precise it passed view-decimals and lost save-model-ref.

5.3 Local models, one at a time

Table 4 compares the local models on the precise and plan prompts, each model served alone with the paired arms sharing the server; cells show the result with minutes (turns

Table 3: *Precise prompt, cloud yardsticks (minutes to finish)*

Model	view-decimals	save-model-ref	log-size	box-press	Passed (median min)
Sonnet 5.5	pass 6.0	pass 7.0	pass 2.0	pass 5.2	4/4 (5.6)
GLM-5.3	pass 13.7	fail 21.7	pass 3.9	pass 19.8	3/4 (16.8)
GLM-5.3-flash	pass 13.9	fail 40.0	pass 4.9	pass 8.3	3/4 (11.1)
Haiku 4.5	fail 8.3	fail 10.6	pass 3.0	pass 8.1	2/4 (8.2)

where recorded). Ornith-1.5-35B-A3B was run in two rounds: round 1 without the levers of Section 5.5, round 2 with the loop breaker, the first-turn retry and the final path hook. Rows of the first, defective serving (no prefix cache) and the two repeat-2 arms killed at the reorder stop are excluded. Ornith ran with thinking off, which is the configuration that is fast enough to use (Section 5.4). Over both rounds Ornith passed 3 of 8 with the precise prompt and 4 of 8 with the plan prompt; Qwen3-Coder-30B (local30) passed 2 of 4 with each. Plan wins view-decimals for every model that ran it (4 of 4 graded arms pass; Ornith twice, local30, Qwen3.8), precise wins box-press for local30 and Ornith. All eight local precise or specification arms on view-decimals failed (local30 four, Ornith two, local30 phase one, Qwen3.8 one), so the localisation hypothesis does not extend to that task. On save-model-ref no local arm passed (0 of 12).

Table 4: *Local models by prompt (night 2; minutes, turns where recorded; 20 or 40 min is the deadline; passed with median wall minutes of graded arms)*

Model	Prompt	view-decimals	save-model-ref	log-size	box-press	Passed (median)
Ornith round 1	precise	fail 20.0	fail 40.0	pass 4.6 / 27	fail 0.1 (empty first turn)	1/4 (12.3)
Ornith round 1	plan	**pass** 20.0	fail 40.0 (no-op loop)	pass 9.5 / 49	fail 25.0	2/4 (22.5)
Ornith with levers	precise	fail 20.0	fail 40.0	pass 4.9 / 25	pass 14.0 / 94	2/4 (17.0)
Ornith with levers	plan	**pass** 20.0	fail 40.0	pass 9.9 / 55	fail 12.5 / 71	2/4 (16.3)
local30	precise	fail 2.6 / 27	fail 31.2 / 52	pass 6.0 / 26	pass 13.3 / 36	2/4 (9.7)
local30	plan	**pass** 10.7 / 36	fail 40.0	pass 9.6 / 29	fail 18.2 / 60	2/4 (14.5)
Qwen3.8-27B	precise	fail 20.0	fail 40.0	pass 15.1	void: pane gate (06:51)	1/3
Qwen3.8-27B	plan	**pass** 20.0	fail 40.0	pass 15.0	void: pane gate (06:51)	2/3
GLM-4.7-Flash	precise	void: pane gate	void: pane gate	void: pane gate[a]	void: pane gate	–

Void: the arm was refused by the worker pane gate (the selected terminal tab belonged to another session) and produced no model result. For GLM-4.7-Flash the server was up at 06:53, but all 8 arms were refused.
[a] A log-size pass of 15.0 minutes exists for GLM-4.7-Flash from 23:18, before the 23:43 reboot, with four servers loaded; pre-reboot, four servers loaded, not comparable, and not counted.

On log-size, the one-function fix, Ornith with the repaired server passed the precise prompt in 4.6 minutes and 27 turns, against 2.0 minutes and 12 turns for Sonnet and 6.2 to 11.6 minutes for local30 on night 1: a local model is within about a factor of two of the cloud reference on a one-function fix.

5.4 Serving findings

Serving decided the outcome again. Table 5 lists the measured defects and what each fix changed.

The multimodal path. Ornith's configuration contains a vision section, so vllm-mlx loaded it through the multimodal batch generator, which never hits the prefix cache; the existing text-forcing patch missed the batched engine. Every turn logged 40–50k remaining prompt tokens with nothing cached, so turns took 15–100 s for 130–2000 output tokens (effective 8–22 tokens/s against 52.8 tokens/s for a single request). Both view-decimals arms ran to the 20-minute deadline without an edit; those rows measure the server, not the model, and are marked as such.

The fold launcher. Two of the new templates reject a system message after the start, so the launcher folds system messages. It appended each turn's trailing system reminder to the last message; the client does not resend it, so the history changed every turn and the cache missed. Keeping only the leading system message fixed it.

Thinking. With thinking on, the chat template drops earlier reasoning from the history, so every turn misses the cache (tested). Thinking was therefore turned off for Ornith and Qwen3.8.

Output cap. Requests carried a maximum of 64,000 output tokens; the new providers now cap output at 8192, as local30 does.

Table 5: *Serving defects of night 2, before and after*

Defect	Before	After
Multimodal path, fold launcher, thinking (Ornith, Qwen3.8)	Prefix cache never hit; 15–100 s per turn; smoke run 55–61 s, 2.1–4.5 s per turn	Cache 11 hit / 1 miss, about 95 % of each prompt cached; smoke run 23–37 s, 0.8–1.5 s per turn
Single-turn speed, shared machine (not a defect)	Ornith: prefill 1679, decode 52.8 tokens/s	Qwen3.8-27B dense: prefill 560, decode 15.2 tokens/s

The three causes were fixed together before the next run, so the table gives their combined effect; separate before and after numbers for each cause were not measured. A fourth fault appeared late and was not fixed: after an aborted request, the Qwen3.8 server rescheduled a 54,445-token prompt in full with no cached prefix, which cost minutes per turn for the dense model (one serving fault more to fix before a dense model is usable).

5.5 Failure classes and levers

Three failure classes recur in the local runs.

Path typo. Absolute paths into the arm's own clone mistyped by a few characters. Lever: the path-repair hook (Section 2). The repair log counts 20 repairs in four Ornith arms (log-size and box-press, round 2), 12 in five Qwen3.8 arms, and none in a cloud or local30 arm; these counts are for arms started after the final hook was registered, and the sample is too small to compare rates between providers.

Repetition loop. Ornith on save-model-ref sent the same Edit with identical old and new strings six or more times in a row ("no changes to make"), every 12 s; the plan arm of the same task looped on a no-op edit until the deadline. Candidate lever: a hook that, after N identical failing calls, returns a stronger message to re-read the file and change approach. It was not applied during the A/B runs.

Empty first turn. Ornith on box-press with the precise prompt answered with five tokens on turn 1 and ended without a tool call; the run was over in six seconds and counts as a fail. Candidate lever: retry in the runner on an empty first turn.

With the levers (Table 4), Ornith passed the same number as before (precise 2 of 4 against 1 of 4, plan 2 of 4 against 2 of 4). On save-model-ref the failure changed from a no-op loop without a build to a build with wrong behaviour, and no pass was gained. The one new pass, box-press with the precise prompt (14.0 minutes, 94 turns), was one where no lever fired (no nudge or denial in its transcript), so it is run-to-run variance and not an effect of the levers.

6 The Reboot

At 23:43 the machine was reset by the hardware watchdog (boot fault record "wdog, reset_in_1"; no kernel panic). The window server's graphics submit thread had been blocked in the kernel graphics driver for 40 s and then 80 s in every one of 12 samples. Four MLX servers were resident, of 33.5, 31.5, 29.3 and 22.6 GB, about 117 GB on a 128 GB machine, with a further 8.6 GB for a document application and 8.2 GB for a virtual machine; the compressor and swap threads were active, but no process was killed, and the thermal level had returned to 0 ten minutes earlier. Our reading is that GPU contention from four servers and about six agents, plus builds, starved the display server, with memory pressure aggravating it. We did not isolate the cause further.

The constraint that followed is itself a finding: on one Apple GPU, the number of servers times the number of concurrent agents is the hard limit, not the amount of RAM. For the rest of the night: at most two servers loaded and later one, weights plus cache under about 75 GB and finally about 35 GB, and at most three local agents at once.

7 Discussion

The prompt matters most for localisable fixes. Naming the file and the function turned two of the four tasks from consistent failure into passes, at about half the turns, which supports the localisation hypothesis of night 1 for those tasks. It did not help where the change itself was hard (view-decimals) or spread over files (save-model-ref). The cloud plan helped on view-decimals for two models, so a divided workflow, cloud head and local hands, is the more promising direction; with these sample sizes it is a hypothesis, not a result.

Serving is a precondition. A model that looks unable to finish may be waiting on re-reading its prompt. Two Ornith arms were lost to a cache defect that was invisible in the pass-fail table. Every future model should be smoke-tested for cache hits before being judged.

Failures are partly mechanical. The path typo, the loop and the empty first turn are deterministic and cheap to guard against, unlike a wrong understanding of the code. What the levers recovered was a failure mode, not passes: no cell changed from fail to pass because of a lever.

8 Recommendation

For local coding beside normal work we recommend Ornith-1.5-35B-A3B at 4 bits, served through the text-path fold launcher with thinking off and the prefix cache on, with one model server and at most two agents at a time, and the path-confinement hook on. Evidence: it was the only local model fast enough for the loop (0.8 to 1.5 s per turn after the cache fix, against up to 190 s for a single Qwen3.8 turn), and it passed one-function fixes within about twice Sonnet's time (log-size 4.6 to 4.9 minutes against 2.0). Weakness: it is level with Qwen3-Coder-30B on passes (7 of 16 against 4 of 8), with one to four runs per cell.

Give it a cloud-written edit plan (cloud head, local hands) for one- and two-function fixes. Evidence: the plan prompt passed view-decimals in 4 of 4 graded arms on three local models, where 0 of 8 precise or specification arms passed. Weakness: one task, and the plan made box-press worse for two models (changes of 43 and 80 lines against 5), so the plan helps only where the change is hard to derive and hurts where it invites a broad edit.

Keep multi-file work, the save-model-ref class, on Sonnet 5.5: it passed in 7.0 minutes, while no local arm passed in 12 tries and GLM-5.3 and GLM-5.3-flash failed it under the precise prompt (n=1 each). Do not use the dense Qwen3.8-27B: 0.9 to 8.1 effective tokens per second with two agents, plus the abort fault above. Haiku 4.5 (2 of 4) fell below GLM-5.3-flash (3 of 4) on these tasks, so GLM-5.3-flash is the cheap cloud fallback (median 11.1 minutes against 8.2 for Haiku); the gap is one task and not significant.

9 Limitations

The samples are small: nine pairs for the central comparison, one to three repeats per cell elsewhere, and a single repeat for Ornith. The sign test cannot reach conventional significance with four discordant pairs. The four tasks come from one subsystem of one code base, and three of them are one-function fixes, which favours the precise prompt. Ornith and Qwen3.8 ran with thinking off, so their results do not describe the models at their best. The GLM-5.3 yardstick ran on a loaded machine; wall times of cloud arms depend on the provider. Runs were censored at the deadline. Prompts and plans were written by Claude models, and a Claude model is also a contestant; grading is mechanical, which limits but does not remove this. Finally, the arms of a pair share one server and the machine's memory, so the absolute speeds are those of a shared server and not of a dedicated one.

References

Anthropic. (2026a). *Claude Code Overview*. [Online]. Available from: https://docs.anthropic.com/en/docs/claude-code/overview (Accessed: October 15, 2026).
(Cit. on p. 4.).

Anthropic. (2026b). *Models Overview: Claude Opus 5.5, Claude Sonnet 5.5 and Claude Haiku 4.5*. [Online]. Available from: https://docs.anthropic.com/en/docs/about-claude/models/overview (Accessed: October 15, 2026).
(Cit. on p. 5.).

Nott, M. (2026a). *AIBroker: A Message Bridge Between Claude Code Sessions and Messaging Channels*. [Online]. Available from: https://github.com/mnott/AIBroker (Accessed: October 15, 2026).
(Cit. on p. 5.).

Nott, M. (2026b). *PAI: Personal AI Infrastructure for Claude Code*. [Online]. Available from: https://github.com/mnott/PAI (Accessed: October 15, 2026).
(Cit. on p. 5.).

Qwen Team. (2025). *Qwen3-Coder-30B-A3B-Instruct*. [Online]. Available from: https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct; 4-bit MLX conversion used: https://huggingface.co/lmstudio-community/Qwen3-Coder-30B-A3B-Instruct-MLX-4bit (Accessed: October 15, 2026).
(Cit. on p. 5.).

vllm-mlx contributors. (2026). *vllm-mlx: OpenAI- and Anthropic-Compatible Serving of MLX Models with Continuous Batching and Prefix Caching*. [Online]. Available from: https://github.com/waybarrios/vllm-mlx (Accessed: October 15, 2026).
(Cit. on p. 4.).

Wilson, E. B. (1927). 'Probable Inference, the Law of Succession, and Statistical Inference', *Journal of the American Statistical Association*, 22 (158), 209–212.
(Cit. on p. 6.).

Z.ai. (2026). *GLM-4.7-Flash*. [Online]. Available from: https://huggingface.co/zai-org/GLM-4.7-Flash (Accessed: October 15, 2026).
(Cit. on p. 5.).